Cloud Fundamentals

Complete Theory Reference Guide

12 Core Sections • 45+ Topics • Easy-to-Read Theory

Table of Contents

01 | Cloud Basics

What is Cloud Computing?

Cloud computing means using someone else's computers (servers, storage, networks) over the internet — instead of buying and managing your own hardware. You pay only for what you use, scale instantly, and access resources from anywhere.

Think of it like electricity. You don't build a power plant — you plug in and pay per unit used.

Cloud Service Models

IaaS	Infrastructure as a Service — You get raw virtual machines, storage, networks. You manage OS, apps, data. Example: AWS EC2, Azure VMs.
PaaS	Platform as a Service — You get a platform to build apps without managing servers. Example: Google App Engine, Heroku, AWS Elastic Beanstalk.
SaaS	Software as a Service — Fully ready-to-use applications. You just use the software. Example: Gmail, Zoom, Salesforce, Office 365.

Public, Private & Hybrid Cloud

Public Cloud	Resources owned and operated by a third-party provider (AWS, Azure, GCP) and shared over the internet. Most cost-effective, highly scalable.
Private Cloud	Cloud infrastructure used exclusively by one organisation, either on-premises or hosted. More control, more cost.
Hybrid Cloud	Mix of public and private cloud — sensitive data stays private, scalable workloads use public. Best of both worlds.

Regions & Availability Zones (AZs)

Cloud providers divide the world into **Regions** (geographic areas like us-east-1, europe-west1). Each region has multiple **Availability Zones** — physically separate data centres within that region. Spreading your app across AZs ensures it keeps running even if one data centre fails.

Concept	What It Is	Why It Matters
Region	A geographic location (e.g. Mumbai, Virginia)	Choose close to users for low latency
Availability Zone	Separate data centre inside a region	Fault isolation — one AZ fails, others run
Edge Location	CDN cache point near end-users	Faster content delivery globally

Shared Responsibility Model

Security in the cloud is a shared job between the cloud provider and you.

Provider's Job	Security OF the cloud — physical hardware, data centres, global network, hypervisor layer. You can't touch this.
Your Job	Security IN the cloud — your data, applications, access controls, OS patches, firewall rules, encryption. This is your responsibility.

02 | Core Cloud Services

Compute Services	Run your code and applications. Includes virtual machines (EC2, Compute Engine), containers (ECS, GKE), and serverless functions (Lambda, Cloud Functions). Choose based on control vs convenience needed.
Storage Services	Store and retrieve data. Object storage for files/blobs (S3, GCS), block storage for disks (EBS, Persistent Disk), file storage for shared drives (EFS, Filestore). Different types suit different use cases.
Networking Services	Connect everything securely. Virtual private clouds (VPC/VNet), load balancers to distribute traffic, DNS to resolve names, CDN for global content delivery, and VPN/Direct Connect for private links.
Managed Databases	Databases where the provider handles installation, patching, backups, and replication. You just query data. Examples: RDS (MySQL/PostgreSQL), DynamoDB (NoSQL), Cloud Spanner, Azure Cosmos DB.
Managed AI/ML Services	Pre-built AI models and ML platforms — no PhD required. Text (GPT/Gemini APIs), Vision (Rekognition), Speech, Translation, and training platforms (SageMaker, Vertex AI) available as simple API calls.

03 | Compute Fundamentals

Virtual Machines (VMs)

A VM is a software-based computer running on physical hardware. It has its own OS, CPU, RAM, and storage — completely isolated from other VMs on the same server. You choose the size (e.g. 2 vCPU / 4 GB RAM) and pay per hour. Full control but you manage the OS.

Containers (Conceptual)

Containers package your application + its dependencies into a lightweight, portable unit. Unlike VMs, containers share the host OS kernel — making them much faster to start and more efficient. Docker is the most popular container tool. Think of a container as a sealed lunchbox — everything it needs is inside.

VM vs Container	VMs: full OS, heavier (GBs), slower start, stronger isolation. Containers: shared OS, lightweight (MBs), instant start, great for microservices.

Auto-Scaling

Automatically adds or removes servers based on traffic or load. **Scale out** = add more servers when traffic spikes. **Scale in** = remove servers when traffic drops. This saves cost and ensures your app handles sudden traffic surges (e.g. a product launch or viral moment).

Load Balancing

A load balancer sits in front of your servers and distributes incoming requests evenly across them. If one server crashes, traffic is routed to healthy ones automatically. It's the traffic cop of your infrastructure.

Serverless Computing

You write a function, upload it — and the cloud runs it only when triggered (an API call, a file upload, a timer). No servers to manage, no idle costs. You pay per invocation (per 100ms of execution). Examples: AWS Lambda, Google Cloud Functions, Azure Functions.

Best for	Event-driven tasks, APIs, data processing, short-lived jobs
Not ideal for	Long-running processes, stateful apps, very latency-sensitive tasks

04 | Storage Fundamentals

■ Object Storage	Store files (images, videos, backups, logs) as objects with a unique key/URL. Infinitely scalable, very cheap, accessed via HTTP. NOT a filesystem — you can't edit in-place, you upload/download whole objects. Examples: AWS S3, Google Cloud Storage, Azure Blob Storage.
■ Block Storage	Raw storage attached to a VM like a hard drive. Fast, low-latency, used for databases and OS disks. Acts exactly like a physical disk — you can format it, partition it, and use it as a file system. Examples: AWS EBS, Azure Managed Disks.
■ File Storage	A shared network drive multiple VMs can mount simultaneously. Useful when many servers need access to the same files. Slower than block, but great for shared workloads. Examples: AWS EFS, Azure Files, Google Filestore.

Type	Use Case	Example Service
Object Storage	Backups, media files, static websites	AWS S3, GCS
Block Storage	Database disks, VM boot volumes	AWS EBS, Azure Disk
File Storage	Shared configs, ML training data	AWS EFS, Azure Files

05 | Networking Basics

Virtual Networks (VPC/VNet)

A Virtual Private Cloud (VPC in AWS/GCP) or Virtual Network (VNet in Azure) is your private, isolated network in the cloud. All your resources live inside it. You control the IP range, routing, and what can talk to what. No traffic enters or leaves unless you explicitly allow it.

Subnets

A subnet is a smaller network carved out of your VPC. Public subnets have internet access; private subnets don't. Use public subnets for web servers, private subnets for databases — databases should never be directly internet-facing.

IP Addressing

Every resource gets an IP address. Private IPs (e.g. 10.0.0.x) are internal only. Public IPs allow internet access. Elastic/Static IPs are fixed public IPs you can assign to a specific resource. CIDR notation (e.g. 10.0.0.0/24) defines an IP range.

Firewalls & Security Groups

Security Groups are virtual firewalls for individual resources (VMs). You define inbound and outbound rules — e.g. allow port 443 from anywhere, allow port 22 only from your office IP. Network ACLs operate at the subnet level and are stateless.

DNS

Domain Name System converts human-readable names (myapp.com) into IP addresses. Cloud DNS services (Route 53, Cloud DNS, Azure DNS) let you manage your domain's records. Key record types: A (name→IP), CNAME (alias), MX (email).

Load Balancers

Distribute incoming traffic across multiple servers to ensure no single server is overwhelmed. Layer 4 (TCP) load balancers are fast. Layer 7 (HTTP) load balancers are smarter — they can route based on URL paths or headers.

06 | Databases in the Cloud

Relational Databases (SQL)

Store data in structured tables with rows and columns. Relationships between tables are defined by keys. Use SQL to query. Best for structured data with complex queries — e.g. user accounts, orders, transactions. ACID compliant — guarantees data integrity.

NoSQL Databases

Flexible databases that don't use traditional tables. Better for unstructured, rapidly changing, or massive-scale data.

Type	Structure	Example Use Case
Key-Value	Simple key → value pairs	Sessions, caching (Redis, DynamoDB)
Document	JSON-like flexible documents	User profiles, product catalogs (MongoDB)
Column-family	Wide columns, sparse data	Analytics, IoT data (Cassandra)
Graph	Nodes and edges	Social networks, recommendations (Neo4j)

Managed Database Services

Instead of installing and managing your own database server, a managed service does it for you. The cloud handles: installation, OS patching, software upgrades, automated backups, monitoring, and failover. You just connect and query.

AWS RDS	Managed MySQL, PostgreSQL, SQL Server, Oracle
AWS DynamoDB	Fully managed NoSQL key-value/document database
Google Cloud SQL	Managed MySQL & PostgreSQL
Azure Cosmos DB	Globally distributed multi-model NoSQL

Backups & Replication

Automated Backups	Cloud takes daily snapshots. You can restore to any point in time.
Read Replicas	Copies of your DB for read-heavy workloads — offload queries from the primary.
Multi-AZ	Standby replica in another AZ. Auto-failover if primary goes down. Zero data loss.

07 | Identity & Security

Identity & Access Management (IAM)

IAM is the system that controls WHO can do WHAT on your cloud resources. It answers: 'Can this user/service delete this S3 bucket?' Every cloud provider has IAM. It's the foundation of cloud security.

Roles & Policies

User	A human identity (person) with credentials (username/password or access keys).
Group	Collection of users. Attach a policy to a group — all members inherit permissions.
Role	An identity for services or apps (not humans). A Lambda function assumes a role to access S3. No static credentials needed.
Policy	A document (JSON) that defines allowed/denied actions. Attached to user, group, or role.

Authentication vs. Authorization

Authentication (AuthN)	Proving who you are. Username + password, MFA, API keys, certificates. 'I am Alice.'
Authorization (AuthZ)	Deciding what you're allowed to do. Policies and permissions. 'Alice can read S3 but not delete EC2.'

Secrets Management

Secrets are sensitive values: database passwords, API keys, certificates. NEVER hardcode them in code or store in environment variables unencrypted. Use a secrets manager — AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault. These encrypt, rotate, and audit access to your secrets.

Network Security Basics

Principle of Least Privilege	Give users/services only the minimum permissions they need. Nothing more.
Zero Trust	Never assume anyone inside the network is trusted. Verify every request.
Encryption in Transit	Use HTTPS/TLS for all data moving across networks.
Encryption at Rest	Encrypt data stored on disks and in databases. Most cloud services offer this by default.

08 | Containers & Orchestration (Awareness)

Docker Fundamentals

Docker is the most popular containerisation tool. It lets you package an application and all its dependencies into a container — a standardised unit that runs identically on any machine. 'Works on my machine' is no longer an excuse.

Dockerfile	A text file with instructions to build an image. FROM, RUN, COPY, CMD.
docker build	Reads a Dockerfile and creates an image.
docker run	Starts a container from an image.
docker-compose	Defines and runs multi-container apps (e.g. app + database together).

Container Images

A container image is a lightweight, read-only snapshot of your application and its environment. Think of it as a template. When you run a container, it creates a writable layer on top of the image. Images are made of layers — each layer is a step in the Dockerfile. Layers are cached, making builds fast.

Container Registries

A registry is a storage and distribution system for container images. You push your built image to a registry, then pull it onto any server to run it.

Docker Hub	Public registry — free for public images, huge library of official images.
AWS ECR	Amazon Elastic Container Registry — private, integrated with ECS/EKS.
Google Artifact Registry	GCP's registry for containers and other artifacts.
Azure Container Registry	Microsoft's private registry, integrates with AKS.

09 | Cloud for AI Workloads

GPU & Accelerator Instances

Training ML models and running large neural networks requires massive parallel computation. GPUs (Graphics Processing Units) are ideal for this — they have thousands of cores for matrix math. Cloud providers offer GPU instances: AWS p4/p5 (NVIDIA A100/H100), GCP A3 (H100), Azure ND-series. TPUs (Tensor Processing Units) are Google's custom ML accelerators — even faster for TensorFlow/JAX workloads.

Batch vs Real-Time Inference

Batch Inference	Process a large dataset all at once, not in real-time. Run overnight, results ready next morning. Use case: monthly churn predictions, image labelling pipelines. Cheaper — can use spot/preemptible instances.
Real-Time Inference	Model responds to a single request instantly (milliseconds). Use case: chatbots, fraud detection, product recommendations. Needs always-on endpoint — more expensive but essential for user-facing features.

Model Deployment Patterns

REST API Endpoint	Deploy model as an HTTP endpoint. Client sends data, gets prediction back. Most common pattern.
Managed Endpoints	SageMaker Endpoints, Vertex AI Endpoints — auto-scale, monitor, A/B test models.
Serverless Inference	Pay-per-call model serving. No idle costs. Slight cold-start latency.
Edge Deployment	Run model on device (phone, IoT sensor). No internet needed. TFLite, ONNX, CoreML.

Data Storage for ML Pipelines

ML pipelines need data at every stage — raw data ingestion, feature stores, model artefacts, experiment tracking.

Raw Data Lake	Object storage (S3/GCS) for raw, unprocessed data in any format.
Feature Store	Centralised store of computed ML features — reusable across models.
Model Registry	Version-controlled store for trained models (MLflow, SageMaker Model Registry).
Experiment Tracking	Log hyperparameters, metrics, results per training run (MLflow, W&B;, Vertex Experiments).

10 | Cost & Resource Management

Pay-As-You-Go Pricing

Cloud pricing is consumption-based — you pay for exactly what you use, by the second/minute/hour. No upfront hardware purchases. This is the fundamental economic shift of cloud vs on-premises.

On-Demand	Full price, no commitment. Use anytime, stop anytime. Best for variable workloads.
Reserved Instances	Commit to 1-3 years upfront — get 40-70% discount. Best for steady workloads.
Spot / Preemptible	Use spare cloud capacity at 60-90% discount. Can be interrupted. Great for batch jobs.
Savings Plans	Commit to a spend amount per hour (not a specific instance) — flexible discount.

Cost Estimation

Before deploying, estimate costs using provider calculators: AWS Pricing Calculator, GCP Pricing Calculator, Azure Pricing Calculator. Input instance types, storage sizes, data transfer, and get a monthly estimate. Always add a buffer — data egress (outbound traffic) costs are often overlooked.

Resource Tagging

Tags are key-value labels you attach to cloud resources. e.g. Environment=Production, Team=DataScience, Project=RecoEngine. Tags let you filter costs by team/project in billing dashboards, enforce policies, and track ownership. **Tag everything from day one** — retrofitting tags is painful.

Budgeting & Cost Monitoring

Budgets & Alerts	Set a monthly budget — get email/SMS alert at 80%, 100%, 120%. AWS Budgets, GCP Budgets.
Cost Explorer	Visualise spending over time, by service, region, or tag. Spot anomalies.
Rightsizing	Identify over-provisioned resources. Downsize instances running at 5% CPU.
Idle Resource Cleanup	Delete unused snapshots, unattached disks, idle load balancers. They still cost money.

11 | DevOps & Automation Basics

Infrastructure as Code (IaC) — Conceptual

Instead of clicking through a UI to create servers, you write code that describes your infrastructure. That code is version-controlled, reviewable, and repeatable. Run it once for dev, once for prod — identical environments.

Terraform	Most popular IaC tool. Cloud-agnostic. Declarative HCL language. Define desired state, Terraform figures out how to get there.
AWS CloudFormation	AWS-native IaC. YAML/JSON templates. Deeply integrated with AWS services.
Pulumi	Write infrastructure using real programming languages (Python, TypeScript).
Ansible	More of a configuration management tool — great for provisioning software on servers.

CI/CD Pipelines — Conceptual

CI (Continuous Integration): Every code commit triggers automated tests. If tests pass, the code is merged. Catches bugs early, maintains code quality.

CD (Continuous Delivery/Deployment): After CI passes, automatically build, package, and deploy to staging (Delivery) or production (Deployment). Goal: code goes from developer laptop to production in minutes, not weeks.

GitHub Actions	CI/CD built into GitHub. YAML workflows triggered by git events.
GitLab CI	Built into GitLab. Powerful pipelines, great for self-hosted.
AWS CodePipeline	AWS-native CI/CD. Integrates with CodeBuild, CodeDeploy.
Cloud Build (GCP)	Managed build service on Google Cloud.

Environment Separation (Dev / Staging / Prod)

Never test in production. Use separate, isolated environments:

Environment	Purpose	Key Rule
Development (Dev)	Developers test new features	Break things freely here
Staging	Mirror of production for final testing	Must match prod config closely
Production (Prod)	Live system serving real users	Changes only via CI/CD. No manual edits.

12 | Cloud Providers Awareness

AWS — Amazon Web Services	Largest cloud provider (~32% market share). Most mature, widest service range (200+ services). Strongest in enterprise, startups, and ML (SageMaker). Regions everywhere. Best ecosystem and third-party tool support.

Compute	EC2 (VMs), ECS/EKS (containers), Lambda (serverless)
Storage	S3 (object), EBS (block), EFS (file), Glacier (archive)
Database	RDS, DynamoDB, Redshift, ElastiCache, Aurora
AI/ML	SageMaker, Bedrock, Rekognition, Comprehend, Polly
Networking	VPC, Route 53, CloudFront, ELB, Direct Connect

GCP — Google Cloud Platform	Google's cloud (~12% share). Best for data analytics (BigQuery), AI/ML (Vertex AI, TPUs), and Kubernetes (they created it). Strong in data engineering workloads. Competitive pricing, especially for sustained use.

Compute	Compute Engine (VMs), GKE (Kubernetes), Cloud Run (serverless containers)
Storage	Cloud Storage (object), Persistent Disk, Filestore
Database	Cloud SQL, Firestore, Bigtable, Spanner, BigQuery
AI/ML	Vertex AI, TPUs, Gemini API, AutoML, Vision/Speech APIs
Networking	VPC, Cloud DNS, Cloud CDN, Cloud Load Balancing

Microsoft Azure	Microsoft's cloud (~23% share). Best for enterprises already using Microsoft products (Windows, Office 365, Active Directory, SQL Server). Strongest hybrid cloud story. Deep government and compliance certifications.

Compute	Azure VMs, AKS (Kubernetes), Azure Functions (serverless)
Storage	Blob Storage, Managed Disks, Azure Files
Database	Azure SQL, Cosmos DB, Azure Database for PostgreSQL
AI/ML	Azure ML, OpenAI Service, Cognitive Services, Bot Service
Networking	VNet, Azure DNS, Azure CDN, Application Gateway, ExpressRoute

Quick Comparison

AWS	GCP	Azure
Market leader, most services	Best ML/data analytics	Best for Microsoft shops
S3, EC2, Lambda icons	BigQuery, Vertex AI, GKE	Active Directory, SQL Server integration
Strongest 3rd-party ecosystem	Lowest cost for sustained use	Best hybrid/on-premises
Best for general workloads	Best for data + AI-first teams	Best for enterprise IT